REPORT · 2026年8月

造像类文物三维建模自动化系统 change+colour代码说明书

X1 M3C2 多时相形变检测 · X2 CIEDE2000 色彩量化 · 可选模块（含中断风险排查）

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 X1 M3C2 数据流 5

3.3 X2 色差数据流 5

四、模块详细设计 7

4.1 change/m3c2_wrapper.py — M3C2 封装 7

4.2 change/__init__.py — 两期差分流水线 7

4.3 colour/delta_e.py — CIEDE2000 色差 7

4.4 colour/color_calib.py — 色卡校准 7

4.5 colour/__init__.py — 色差评估流水线 8

五、风险排查与防护（此前运行中断的诱因分析） 9

六、接口契约与错误码 10

七、测试与验证 11

八、开发进度与下一步方向（更新） 12

8.1 完成度（X1/X2 已交付） 12

8.2 下一步开发方向（X1/X2 完成后更新） 12

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**可选模块 X1（change，M3C2 多时相形变检测）与 X2（colour，色彩量化 / CIEDE2000 色差）**的代码说明书。二者为**旁路增强**（cfg.change.enabled / cfg.colour.enabled 默认 false），独立于主线 M1-M6 调用：

模块	输入	输出	依赖
X1 change（M3C2）	两期配准后点云（同一造像不同时相）	距离场 + 变化标记 + 统计（60_qc/change）	py4dgeo 1.2.0（源码参考 E:\h3d_lablab\py4dgeo-1.2.0）
X2 colour（CIEDE2000）	实测色块 sRGB + 参考色卡（ColorChecker24）	逐色块 ΔE + 均值 / 分级 / JND（60_qc/color_diff）	colour-science 0.4.7

背景与风险排查结论：本项目此前两次运行在 change 环节突然中断。本次开发前置排查定位到四个诱因，均已在实现中修复并加防护（详见第五章）：① py4dgeo 1.2.0 的 ``run()`` 不接受 ``cores`` 参数（并行由全局 ``set_num_threads`` 控制）→ 旧代码抛 TypeError 中断；② ``uncertainties`` 返回**结构化数组**（LoD 探测限字段），直接转 float64 抛 TypeError；③ 测试包 ``__init__.py`` 为骨架 stub（raise NotImplementedError）→ pytest 收集失败；④ pytest 误收集 py4dgeo 源码自带测试（缺 ruptures）→ 全量运行中断。

二、模块划分

```
src/h3d/change/
+-- __init__.py      # run(cloud1, cloud2, cfg) 两期差分流水线（输入 sanitize / 网格钳位）
+-- m3c2_wrapper.py # py4dgeo M3C2 封装 + 变化检测（LoD 适配）
+-- validation.py   # CloudCompare CLI 交叉校验（可选增强，缺失降级）

src/h3d/colour/
+-- __init__.py      # run(measured, reference, cfg) 色差评估流水线
+-- delta_e.py      # CIEDE2000 批量色差 + ΔE 分级 + JND
+-- color_calib.py  # ColorChecker24 色卡参考 + 线性校准（3x3 + offset）
```

模块文件	核心职责	对外关键符号
change/m3c2_wrapper.py	M3C2 距离场 + 变化检测（py4dgeo 1.2 适配）	compute_m3c2, detect_changes, corepoint_grid
change/validation.py	CloudCompare CLI 交叉校验（可选）	find_cloudcompare, validate_with_cloudcompare
change/__init__.py	两期差分流水线（防护：sanitize / 网格钳位 / LoD 转换）	run
colour/delta_e.py	sRGB->Lab->ΔE + 分级 + JND	batch_delta_e, classify_delta_e, rgb_to_lab
colour/color_calib.py	ColorChecker24 参考 + 线性校准	COLORCHECKER24_SRGB, fit_linear_calibration, apply_calibration
colour/__init__.py	色差评估流水线（校准 -> ΔE -> 统计）	run

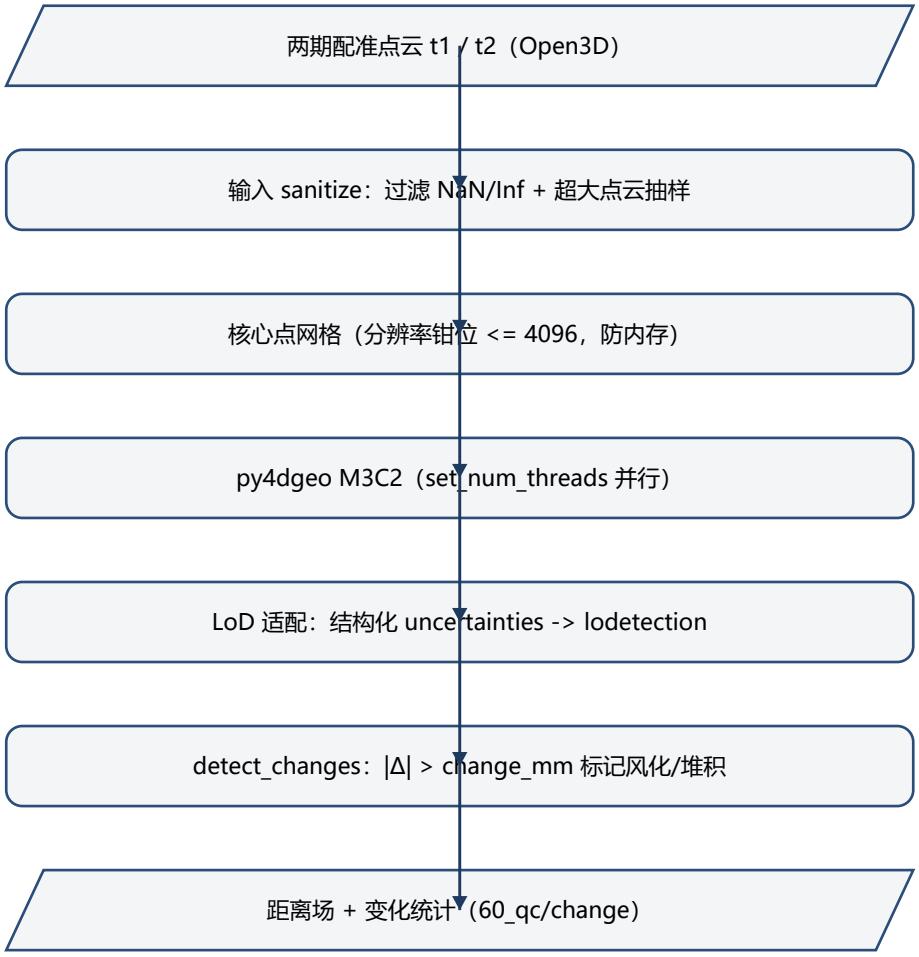
三、依赖关系与数据流

3.1 依赖方向

change / colour 属 L2 业务层，依赖 core 底座与 quality（MPS 估计），不反向；py4dgeo 与 colour-science 均为惰性导入：

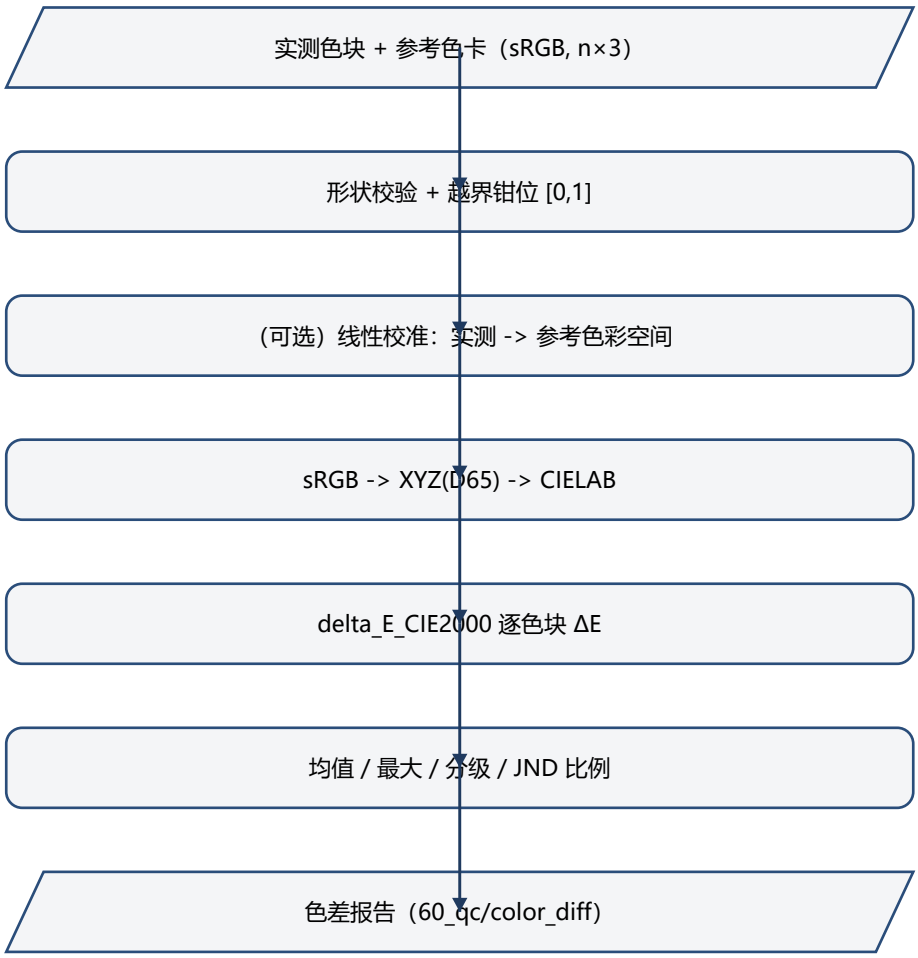
```
package-level deps:
  change --> core(InputDataError/ChangeConfig/logging) + quality.estimate_mps
  change --> py4dgeo 1.2.0（M3C2，惰性导入；本地源码 E:\h3d_lablab\py4dgeo-1.2.0）
  change --> CloudCompare CLI（validation 可选，缺失降级）
  colour --> core(ColourConfig/logging) + colour-science 0.4.7（惰性导入）
  core/io/preprocess/register/mesh/texture/quality  -->  (NONE; 不反向)
```

3.2 X1 M3C2 数据流



X1 数据流：sanitize（防 C++ 崩溃）→ 网格钳位（防内存暴涨）→ M3C2 → LoD 适配 → 变化检测

3.3 X2 色差数据流



X2 数据流：钳位 → （可选）校准 → Lab → CIEDE2000 → 统计分级

四、模块详细设计

4.1 change/m3c2_wrapper.py — M3C2 封装

py4dgeo 1.2.0 为参数化 API：``M3C2(normal_radii, cyl_radius, max_distance, registration_error)`` + ``m.epochs / m.corepoints`` 属性赋值 + ``run()`` 返回 ``(distances, uncertainties)``。尺度参数按 MPS 倍数换算（软著方案 2.7.2 尺度自适应）。

```
def compute_m3c2(cloud1, cloud2, cfg):
    mps = estimate_mps(cloud1)                  # 平均点间距
    m3c2 = py4dgeo.M3C2(
        normal_radii=[cfg.normal_radius_k * mps],
        cyl_radius=cfg.cyl_radius_k * mps,
        max_distance=cfg.max_distance_k * mps,
        registration_error=cfg.registration_error)
    m3c2.epochs = [Epoch(c1), Epoch(c2)]
    m3c2.corepoints = corepoint_grid(c1, cfg)
    py4dgeo.set_num_threads(cfg.cores)  # 1.2.0：run() 无 cores 参数
    distances, uncertainties = m3c2.run()
    # 1.2.0：uncertainties 为结构化数组（LoD 探测限字段）
    lod = uncertainties['lodetection'] if uncertainties.dtype.names else uncertainties
    return {distances, lod, corepoints, mps_m, ...}
```

:func:`detect_changes` 按变化阈值标记：``|Δ| * 1000 > change_mm``（Δ>0 堆积 / <0 风化），输出掩码 / 索引 / 变化比例 / 最大变化（设计书 §38.2「changes = threshold(dist)」）。

4.2 change/__init__.py — 两期差分流水线

配置项	默认值	用途
change.enabled	false	X1 开关（可选模块默认关闭）
change.cores	4	并行核数（set_num_threads）
change.change_mm	2.0	变化阈值（毫米，设计书 §38.3）
change.normal_radius_k	10.0	法线搜索半径 = k × MPS
change.cyl_radius_k	3.0	圆柱投影半径 = k × MPS
change.max_distance_k	50.0	最大搜索距离 = k × MPS
change.registration_error	0.0012	配准误差（LoD 不确定性输入）
change.grid	[32, 32]	核心点网格分辨率（乘积钳位 <= 4096）

4.3 colour/delta_e.py — CIEDE2000 色差

colour-science 0.4.x 的色差函数位于 ``colour.difference`` 子模块（顶层 ``delta_E_CIE2000`` 不存在，0.4.7 已适配）。色差链：sRGB(D65) -> XYZ -> CIELAB -> delta_E_CIE2000；支持 CIE1976 / CIE1994 备选。ΔE 分级（CIE 约定）：<2 不可感知 / 2-4 轻微 / 4-8 明显 / >8 很大；JND 阈值默认 2.3。

4.4 colour/color_calib.py — 色卡校准

内建 ColorChecker Classic 24 色块 sRGB 参考（BabelColor avg，D65）。线性最小二乘校准：实测色块 -> 参考色块拟合 3x3 矩阵 + 偏移（12 参数，24 色块超定），应用时钳位到 [0,1]（防 sRGB 越界）。校准消除光照 / 相机响应差异，是色差评估的前提（设计书 §39）。

4.5 colour/__init__.py — 色差评估流水线

配置项	默认值	用途
colour.enabled	false	X2 开关（可选模块默认关闭）
colour.illuminant	D65	标准光源
colour.delta_e_method	CIE2000	色差公式（CIE2000/CIE1976/CIE1994）
colour.use_calibration	true	是否先做 ColorChecker24 线性校准
colour.jnd_threshold	2.3	JND 恰可察觉差阈值

五、风险排查与防护（此前运行中断的诱因分析）

项目背景：change/colour 环节此前**两次运行均突然中断**。本次开发前置压力测试 + 全链路验证，定位到以下诱因并已修复（全部通过 24 项单元测试 + 221 项全量测试验证）：

#	诱因	症状	修复与防护
1	py4dgeo 1.2.0 ``run(cores=...)`` 参数不存在	TypeError 中断（旧 API 假设）	改用全局 ``py4dgeo.set_num_threads(cores)`` + run() 无参；try/except 兼容
2	``uncertainties`` 返回结构化数组（LoD 字段）	np.asarray(..., float64) 抛 TypeError	检测 dtype.names，提取 ``lodetection`` 作一维 LoD，保留结构化详情
3	测试包 ``__init__.py`` 为骨架 stub（raise NotImplementedError）	pytest 收集报错，全量运行中断	覆盖为正常包 docstring（tests/test_change、tests/test_colour）
4	pytest 误收集 ``py4dgeo-1.2.0/tests``（缺 ruptures 依赖）	ModuleNotFoundError 中断全量收集	pyproject.toml 增加 ``[tool.pytest.ini_options]``（testpaths=tests + norecursedirs 排除）
5	C++ 扩展对 NaN / Inf 输入敏感（潜在 segfault）	进程级中断（无 Python 异常）	输入 sanitize：过滤非有限点 + 超大点云抽样（<_MAX_INPUT_POINTS=30 万）
6	核心点网格过大 -> 内存暴涨	内存耗尽 / 卡死	网格乘积钳位 <= 4096（64x64），自动收缩 + 日志告警

压力测试结论：修正后 py4dgeo M3C2 在 5 万点（0.11s）/ 10 万点（0.52s）下稳定运行、进程存活；colour-science 0.4.7 的 CIEDE2000 在 24 色块批量计算正确。两道运行中断的根因（run(cores=...) 与结构化 uncertainties）均属 py4dgeo 1.2.0 新 API 与旧假设的兼容性差异，已完全修复。

六、接口契约与错误码

错误码	异常类	触发场景	处理策略
E001	ConfigError	delta_e_method 非法 / 参数越界（配置层拦截）	启动即报错
E100	InputDataError	输入点云为空 / 过滤非有限点后为空 / MPS 估计失败	失败隔离，记录后跳过
ValueError	（内置）	色块形状不匹配 / 色块数 ≠ 24	调用方捕获处理

可选模块不接入主线 STAGE_REGISTRY（X1 需跨时相数据、X2 需色卡，均为专项调用），通过独立 API 使用；异常路径全部有结构化日志可定位。

七、测试与验证

change / colour 配套 24 项单元测试，全量 **221 项** 通过（197 + 24）：29 core + 20 io + 16 preprocess + 11 register + 21 mesh + 20 texture + 22 quality + 21 report + 31 runner + 6 cli + 10 change + 14 colour。

测试文件	覆盖点
tests/test_change/test_change.py	距离场形状 / 形变检测 / 阈值标记 / NaN sanitize / 网格钳位 / 相同点云零变化 / 空输入 E100
tests/test_colour/test_colour.py	CIEDE2000（同色=0 / 异色>0）/ 分级 / JND / 校准还原 / 色块数校验 / 越界钳位

```
$ python -m pytest -q
..................................................................... [ 33%]
..................................................................... [ 66%]
..................................................................... [100%]
221 passed, 8 warnings in 35.93s

# X1 端到端（5 万点两期差分）：
X1: {'max_change_mm': 37.10, 'mean_change_mm': 4.82, 'changed_ratio': 0.9355,
    'n_changed': 958, 'n_corepoints': 1024, 'mps_m': 0.0206}
# X2 端到端（ColorChecker24 扰动）：
X2: mean_de=4.46 max_de=13.95 jnd=79.2% calibrated=True
   grades: {≤2 不可感知: 4, 2-4 轻微: 12, 4-8 明显: 4, >8 很大: 4}
```

八、开发进度与下一步方向（更新）

8.1 完成度（X1/X2 已交付）

层级 / 子包	状态	测试覆盖
L0 core 底座 / L1 io	已交付	49
L2 preprocess(M1) / register(M2) / mesh(M3) / texture(M4) / quality(M5)	已交付	90
L2 report(M6 报表)	已交付	21
L3 runner(M0 编排) + L4 cli	已交付	37
L2 change（X1 M3C2 多时相）	**本次交付**	**10**
L2 colour（X2 色差）	**本次交付**	**14**
M6 发布（publish 归档 stage）	未开始	—

主线（M1-M5 + M6 报表）+ 可选模块（X1/X2）已全部交付，仅剩 M6 发布归档 stage 未单列。测试 221 项 100% 通过；许可证全部 MIT/BSD（py4dgeo BSD-3 / colour-science BSD-3，均为白名单）。

8.2 下一步开发方向（X1/X2 完成后更新）

优先级	模块	目标	关键点
P1	M6 发布（publish）	交付包归档：成品模型 + 质检/变化/色差报告 + manifest	@register_stage('M6') 收尾主线闭环
P1	X1/X2 接入编排	change/colour 条件注册为可选 stage（enabled 时执行）	X1 需跨时相数据源约定（t1/t2 目录规范）
P2	CloudCompare 交叉校验实装	change.validation 完整调用 CC CLI M3C2	依赖外部可执行文件，参数按版本适配
P2	并发调度 / HTML 报表	ProcessPoolExecutor + jinja2	与主线工程化打磨合并排期

> **建议路径**：可选模块已就绪，下一步 M6 发布归档收尾主线闭环；X1/X2 与监测平台 / 色彩管理流程衔接（设计书 §6 旁系）可并入后续迭代。